

合(least squares fit), 我们发现, psum1 和 psum2 的运行时间(用时钟周期为单位)分别近似于等式 $368 + 9.0n$ 和 $368 + 6.0n$ 。这两个等式表明对代码计时和初始化过程、准备循环以及完成过程的开销为 368 个周期加上每个元素 6.0 或 9.0 周期的线性因子。对于较大的 n 的值(比如说大于 200), 运行时间就会主要由线性因子来决定。这些项中的系数称为每元素的周期数(简称 CPE)的有效值。注意, 我们更愿意用每个元素的周期数而不是每次循环的周期数来度量, 这是因为像循环展开这样的技术使得我们能够用较少的循环完成计算, 而我们最终关心的是, 对于给定的向量长度, 程序运行的速度如何。我们将精力集中在减小计算的 CPE 上。根据这种度量标准, psum2 的 CPE 为 6.0, 优于 CPE 为 9.0 的 psum1。

旁注 什么是最小二乘拟合

对于一个数据点 $(x_1, y_1), \dots, (x_n, y_n)$ 的集合, 我们常常试图画一条线, 它能最接近于这些数据代表的 $X-Y$ 趋势。使用最小二乘拟合, 寻找一条形如 $y = mx + b$ 的线, 使得下面这个误差度量最小:

$$E(m, b) = \sum_{i=1, n} (mx_i + b - y_i)^2$$

将 $E(m, b)$ 分别对 m 和 b 求导, 把两个导数函数设置为 0, 进行推导就能得出计算 m 和 b 的算法。

练习题 5.2 在本章后面, 我们会从一个函数开始, 生成许多不同的变种, 这些变种保持函数的行为, 又具有不同的性能特性。对于其中三个变种, 我们发现运行时间(以时钟周期为单位)可以用下面的函数近似地估计:

版本 1: $60 + 35n$

版本 2: $136 + 4n$

版本 3: $157 + 1.25n$

每个版本在 n 取什么值时是三个版本中最快的? 记住, n 总是整数。

5.3 程序示例

为了说明一个抽象的程序是如何被系统地转换成更有效的代码的, 我们将使用一个基于图 5-3 所示向量数据结构的运行示例。向量由两个内存块表示: 头部和数据数组。头部是一个声明如下的结构:

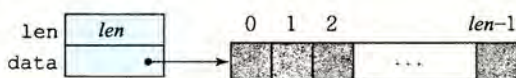


图 5-3 向量的抽象数据类型。向量由头信息加上指定长度的数组来表示

code/opt/vec.h

```
1  /* Create abstract data type for vector */
2  typedef struct {
3      long len;
4      data_t *data;
5  } vec_rec, *vec_ptr;
```

code/opt/vec.h

这个声明用 `data_t` 来表示基本元素的数据类型。在测试中, 我们度量代码对于整数(C 语言的 `int` 和 `long`)和浮点数(C 语言的 `float` 和 `double`)数据的性能。为此, 我们会分别为不同的类型声明编译和运行程序, 就像下面这个例子对数据类型 `long` 一样:

```
typedef long data_t;
```

我们还会分配一个 `len` 个 `data_t` 类型对象的数组，来存放实际的向量元素。

图 5-4 给出的是一些生成向量、访问向量元素以及确定向量长度的基本过程。一个值得注意的重要特性是向量访问程序 `get_vec_element`，它会对每个向量引用进行边界检查。这段代码类似于许多其他语言(包括 Java)所使用的数组表示法。边界检查降低了程序出错的机会，但是它也会减缓程序的执行。

```

code/opt/vec.c

1  /* Create vector of specified length */
2  vec_ptr new_vec(long len)
3  {
4      /* Allocate header structure */
5      vec_ptr result = (vec_ptr) malloc(sizeof(vec_rec));
6      data_t *data = NULL;
7      if (!result)
8          return NULL; /* Couldn't allocate storage */
9      result->len = len;
10     /* Allocate array */
11     if (len > 0) {
12         data = (data_t *)calloc(len, sizeof(data_t));
13         if (!data) {
14             free((void *) result);
15             return NULL; /* Couldn't allocate storage */
16         }
17     }
18     /* Data will either be NULL or allocated array */
19     result->data = data;
20     return result;
21 }
22
23 /*
24  * Retrieve vector element and store at dest.
25  * Return 0 (out of bounds) or 1 (successful)
26  */
27 int get_vec_element(vec_ptr v, long index, data_t *dest)
28 {
29     if (index < 0 || index >= v->len)
30         return 0;
31     *dest = v->data[index];
32     return 1;
33 }
34
35 /* Return length of vector */
36 long vec_length(vec_ptr v)
37 {
38     return v->len;
39 }

```

code/opt/vec.c

图 5-4 向量抽象数据类型的实现。在实际程序中，数据类型 `data_t` 被声明为 `int`、`long`、`float` 或 `double`

作为一个优化示例，考虑图 5-5 中所示的代码，它使用某种运算，将一个向量中所有的元素合并成一个值。通过使用编译时常数 IDENT 和 OP 的不同定义，这段代码可以重编译成对数据执行不同的运算。特别地，使用声明：

```
#define IDENT 0
#define OP +
```

它对向量的元素求和。使用声明：

```
#define IDENT 1
#define OP *
```

它计算的是向量元素的乘积。

```
1  /* Implementation with maximum use of data abstraction */
2  void combinel(vec_ptr v, data_t *dest)
3  {
4      long i;
5
6      *dest = IDENT;
7      for (i = 0; i < vec_length(v); i++) {
8          data_t val;
9          get_vec_element(v, i, &val);
10         *dest = *dest OP val;
11     }
12 }
```

图 5-5 合并运算的初始实现。使用基本元素 IDENT 和合并运算 OP 的不同声明，我们可以测量该函数对不同运算的性能

在我们的讲述中，我们会对这段代码进行一系列的变化，写出这个合并函数的不同版本。为了评估性能变化，我们会在一个具有 Intel Core i7 Haswell 处理器的机器上测量这些函数的 CPE 性能，这个机器称为参考机。3.1 节中给出了一些有关这个处理器的特性。这些测量值刻画的是程序在某个特定的机器上的性能，所以在其他机器和编译器组合中不保证有同等的性能。不过，我们把这些结果与许多不同编译器/处理器组合上的结果做了比较，发现也非常相似。

我们会进行一组变换，发现有很多只能带来很小的性能提高，而其他的能带来更巨大的效果。确定该使用哪些变换组合确实是编写快速代码的“魔术(black art)”。有些不能提供可测量的好处的组合确实是无效的，然而有些组合是很重要的，它们使编译器能够进一步优化。根据我们的经验，最好的方法是实验加上分析：反复地尝试不同的方法，进行测量，并检查汇编代码表示以确定底层的性能瓶颈。

作为一个起点，下表给出的是 combinel 的 CPE 度量值，它运行在我们的参考机上，尝试了操作(加法或乘法)和数据类型(长整数和双精度浮点数)的不同组合。使用多个不同的程序，我们的实验显示 32 位整数操作和 64 位整数操作有相同的性能，除了涉及除法操作的代码之外。同样，对于操作单精度和双精度浮点数据的程序，其性能也是相同的。因此在表中，我们将只给出整数数据和浮点数据各自的结果。

函数	方法	整数		浮点数	
		+	*	+	*
combinel	抽象的未优化的	22.68	20.02	19.98	20.18
combinel	抽象的-O1	10.12	10.12	10.17	11.14

可以看到测量值有些不太精确。对于整数求和的 CPE 数更像是 23.00，而不是 22.68；对于整数乘积的 CPE 数则是 20.0 而非 20.02。我们不会“捏造”数据让它们看起来好看一点儿，只是给出了实际获得的测量值。有很多因素会使得可靠地测量某段代码序列需要的精确周期数这个任务变得复杂。检查这些数字时，在头脑里把结果向上或者向下取整几百分之一个时钟周期会很有帮助。

未经优化的代码是从 C 语言代码到机器代码的直接翻译，通常效率明显较低。简单地使用命令行选项“-O1”，就会进行一些基本的优化。正如可以看到的，程序员不需要做什么，就会显著地提高程序性能——超过两个数量级。通常，养成至少使用这个级别优化的习惯是很好的。（使用-Og 优化级别能得到相似的性能结果。）在剩下的测试中，我们使用-O1 和-O2 级别的优化来生成和测量程序。

5.4 消除循环的低效率

可以观察到，过程 combine1 调用函数 vec_length 作为 for 循环的测试条件，如图 5-5 所示。回想关于如何将含有循环的代码翻译成机器级程序的讨论（见 3.6.7 节），每次循环迭代时都必须对测试条件求值。另一方面，向量的长度并不会随着循环的进行而改变。因此，只需计算一次向量的长度，然后在我们的测试条件中都使用这个值。

图 5-6 是一个修改了的版本，称为 combine2，它在开始时调用 vec_length，并将结果赋值给局部变量 length。对于某些数据类型和操作，这个变换明显地影响了某些数据类型和操作的整体性能，对于其他的则只有很小甚至没有影响。无论是哪种情况，都需要这种变换来消除这个低效率，这有可能成为尝试进一步优化时的瓶颈。

```
1  /* Move call to vec_length out of loop */
2  void combine2(vec_ptr v, data_t *dest)
3  {
4      long i;
5      long length = vec_length(v);
6
7      *dest = IDENT;
8      for (i = 0; i < length; i++) {
9          data_t val;
10         get_vec_element(v, i, &val);
11         *dest = *dest OP val;
12     }
13 }
```

图 5-6 改进循环测试的效率。通过把对 vec_length 的调用移出循环测试，我们不再需要每次迭代时都执行这个函数

函数	方法	整数		浮点数	
		+	*	+	*
combine1	抽象的-O1	10.12	10.12	10.17	11.14
combine2	移动 vec_length	7.02	9.03	9.02	11.03

这个优化是一类常见的优化的一个例子，称为代码移动(code motion)。这类优化包括识别要执行多次(例如在循环里)但是计算结果不会改变的计算。因而可以将计算移动到代码前面不会被多次求值的部分。在本例中，我们将对 vec_length 的调用从循环内部移动